Qiskit 量子回路コードの修正介入と救済対象の分析

2023TC011　伊藤 悠真
指導教員　横山 哲郎

実行結果提示と自己デバッグを，同じ初期失敗から比較する

動くコードでも，期待した出力とは限らない

読込み

import ・ API の失敗

回路構築

回路を作れない

出力検証

期待分布と不一致

修正でどれだけ救済できるか
同じ失敗を救済するのか

背景： Self-Refine [1] ／ Self-Debugging [2] （完全再現ではない）

同じ初期コードから，３種類の修正を比較

５モデル　× 10 タスク　× seed 0 ～ 19　＝　1,000 初期組

初期生成　B

→ R　自己見直し

→ F　実行結果提示

→ D　診断＋説明・原因分析

最大３修正・成功時停止　／　１要求 1,024 tokens　／　P は独立生成

救済率の分母は，初期失敗 545 組

1,000 初期組 ＝ 初期成功 455 組 ＋ 初期失敗 545 組

救済率 ＝ 初期失敗から最終成功となった組数 ／ 545

成功: 読込み・構築・指定入力の期待出力・構造要件
全入力での正しさを保証する評価ではない

検証済み保存 prefix ＋必要な部分追試 suffix による現行経路

F と D は初期失敗の約 3 分の 1 を救済

共通の分母：初期失敗 545 組



今回の対象での観測．モデル・介入の一般的優劣は断定しない．

救済数が近くても，救済する対象は異なる

43 組	140 組	40 組
F のみ	F/D 共通	D のみ

片側救済 83 組の全履歴を確認

両条件 166 履歴・ 398 修正遷移 ／ 双方未救済 322 組

複数箇所が同時に変わるため，成功原因は分離できない

3 回目にも救済がある．追加消費と併せて考える

追加 tokens ／救済

R	18,852.6
F	9,333.8
D	10,976.6

3 回目の追加救済

F	28 組
D	33 組

2 回で止めると
F15.3% ・ D18.3% の救済を失う

共有初期を除き未救済消費を含む計算値．同じ最大 3 修正の履歴を区切った比較．

救済率に加え，「何を救済するか」が重要

1　F183 組・D180 組を救済（初期失敗 545 組）

2　片側救済 83 組：介入で救済対象が異なる

3　追加消費と停止回数の両面から検討する

今後：選択・停止方策を事前固定し，別 seed ・タスクで検証

補足 A　モデル・初期失敗類型で傾向が変わる

モデル	失敗分母	R	F	D
Q4	162	31	78	90
G4	62	30	48	38
Q9	66	15	33	28
C7	65	15	23	23
G3	190	5	1	1

初期 import 失敗（271 組）

F 58 組 ／ D 72 組

初期回路構築失敗（126 組）

F 76 組 ／ D 68 組

Q4/G4/Q9/C7/G3 の正式タグは要旨・論文本体のモデル表を参照.

補足 B　評価と経路の範囲

主 L3：指定入力の出力＋所定の構造要件

分布：8,192 shots ・全変動距離 0.05 未満

検証済み保存 prefix ＋部分追試 293 ケースの suffix

旧 120 件は合算しない／P は独立生成

共有初期・タスク・seed 依存，診断文・負荷差が残る

P：改善 124 組・悪化 106 組，最終成功 473/1,000 組

補足 C　参考文献・固定出典

[1] Madaan et al., Self-Refine, arXiv:2303.17651v2 (2023).

[2] Chen et al., Teaching Large Language Models to Self-Debug,
arXiv:2304.05128v2 (2023).

[3] 現行救済経路の固定集計（2026-10-16 版）
docs/presentation_sources/20261016T_reachable_rescues/

固定 snapshot ・全分母 CSV ・ sanity/validation report を使用 .
本資料の作成に伴う追加実験は行っていない .